UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Data representation and number systems

Answer: -20.12510 = 0xC1A10000 (sign=1, exponent=10000011, fraction=01000010000000000000000)

- *1 mark for the sign bit.*
- *1 mark for the biased exponent.*
- *1 mark for the 23-bit fraction.*
- *1 mark for the correct 32-bit grouping.*
- *1 mark for the final hexadecimal value.*

QUESTION A2 (5 marks)

Topic: Computer generations and history

Source: CSC429 (3).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The Step Reckoner (also called the Leibniz wheel or stepped reckoner) was invented by Gottfried Wilhelm Leibniz. The Mechanical Calculating Machine referred to in the statement is the Pascaline (Pascal's calculator), invented by Blaise Pascal.

(b) The main functional difference is that Pascal's machine could only perform addition and subtraction (directly), whereas Leibniz's Step Reckoner could perform all four basic arithmetic operations: addition, subtraction, multiplication and division. The Step Reckoner achieved multiplication and division through repeated addition/subtraction using its stepped drum mechanism, while the Pascaline used a single-tooth carry mechanism limited to add/subtract.

(c) The Step Reckoner (Leibniz's machine) is the one associated with the binary numbering system; Leibniz developed the binary system and incorporated binary principles in the design of his calculating machine, which is fundamental to modern computers.

Unofficial marking guide:

- *1 mark: Correctly names Gottfried Wilhelm Leibniz as inventor of the Step Reckoner.*
- *1 mark: Correctly names Blaise Pascal as inventor of the Mechanical Calculating Machine (Pascaline).*
- *1 mark: States that Pascal's machine performed only addition and subtraction.*
- *1 mark: States that Leibniz's Step Reckoner performed all four arithmetic operations (addition, subtraction, multiplication, division).*
- *1 mark: Identifies the Step Reckoner (Leibniz's machine) as using the binary numbering system.*

Model confidence: medium.

Model note: The question text refers to 'Mechanical Calculating Machine' generically; it is assumed to mean the Pascaline (Pascal's calculator) based on standard computer history. The Step Reckoner is also known as the Leibniz wheel. The binary association with the Step Reckoner is historically debated: Leibniz designed the machine with a stepped drum (decimal), but he also developed binary arithmetic and envisioned binary machines; the expected answer in this context is the Step Reckoner.

QUESTION A3 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (10).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) One characteristic of SMP: all processors share a single common main memory (and typically the system bus/I/O subsystem), so each processor has equal access to the same physical memory and there is a single copy of the operating system managing all processors.

(b) Meaning of "symmetric": the word symmetric means that all processors are treated as peers — they are identical/equal in capability, each has equal access to shared memory and I/O, and no processor is designated as a master or controller. Any processor can execute any task, including OS kernel code, and the workload can be distributed evenly among them; the OS sees one pool of processors rather than a hierarchy of master and subordinate (slave) processors.

(c) One advantage of SMP organization: it gives good scalability/throughput and graceful degradation — additional processors can be added to increase performance, and if one processor fails the remaining processors can continue running the system (fault tolerance), while a single OS image makes the system

© Hak Cipta Universiti Teknologi MARA

easier to manage and keeps inter-processor communication through shared memory relatively fast.

Unofficial marking guide:

• 1 mark: correct characteristic of SMP (e.g. shared common main memory / single OS image / equal access to memory and I/O).

• 1 mark: states that all processors are equal/identical peers with no master-slave distinction.

• 1 mark: explains the consequence — any processor can run any task including OS code, equal access to shared memory/I/O, workload shared evenly.

• 1 mark: correct advantage stated (e.g. scalability/throughput, graceful degradation/fault tolerance, or ease of management with single OS).

• 1 mark: brief justification/explanation of why it is an advantage.

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. Mark allocation follows the subpart totals (1 + 2 + 2 = 5). This is an unofficial model answer, not an official marking scheme.

QUESTION A4 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the effective address of the operand itself, so the operand is fetched directly from memory location A — the address field points straight at the operand, with no intermediate pointer.

(b) How it is used: the CPU decodes the instruction, takes the address field A as the effective address (EA = A), and issues a memory read to that location. The operand stored at memory address A is then loaded into the CPU (e.g. into a register or the accumulator) for the operation. In the figure, the instruction's Address A field is used as the memory address, and the memory at that address holds the operand. No further address calculation or indirection is performed; the address field is used unchanged as the operand's location.

(c) Advantage: it is simple and fast — only one memory access is needed to fetch the operand (no extra indirection), and no address computation is required. Disadvantage: the address field must be large enough to hold a full memory address, limiting the range of addressable memory for a fixed instruction length; it is also inflexible (the address is fixed in the instruction, so it cannot easily support relocation or pointer-based structures).

Unofficial marking guide:

• 1 mark: correctly labels the technique as direct addressing (accept absolute addressing).

• 1 mark: states that the address field A is used directly as the effective address (EA = A).

• 1 mark: explains the operand is fetched from memory at address A in a single memory access (no indirection).

• 1 mark: gives one valid advantage (e.g. simple/fast, single memory access, no address calculation).

• 1 mark: gives one valid disadvantage (e.g. limited addressable range due to address field size, inflexibility/no relocation or pointer support).

Model confidence: high.

Model note: Figure 2 is inferred from the extracted text: an instruction with Opcode and Address A, and memory containing the operand at that address. This layout corresponds to direct addressing; if the figure actually shows the address field pointing to a memory word that itself contains the operand's address, the answer would instead be indirect addressing. The subpart (b) reference to '9(a)' is treated as a typo for 'a)'.

QUESTION A5 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (8).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Complete machine cycle with the exact location of each phase. The machine cycle is the sequence of operations the CPU performs to fetch, decode, and execute one instruction (and, where relevant, store the result). Draw it as a cycle of phases, each phase located in the unit where it happens:

[1] FETCH phase — located in the CPU, specifically the Control Unit (CU) together with the Program Counter (PC), Memory Address Register (MAR), Memory Data Register (MDR) and main memory. The PC holds the address of the next instruction; this address is copied to the MAR, placed on the address bus, and the instruction is read from main memory into the MDR, then transferred to the Instruction Register (IR). The PC is then incremented to point to the next instruction.

© Hak Cipta Universiti Teknologi MARA

[2] DECODE phase — located in the Control Unit (instruction decoder). The opcode in the IR is decoded to determine which operation is required and which operands/registers are involved; the CU generates the corresponding control signals.

[3] EXECUTE phase — located in the Arithmetic Logic Unit (ALU) and/or registers, under control of the Control Unit. The required operation (arithmetic, logic, data movement, branch, etc.) is carried out; operands come from registers or memory and the result is produced.

[4] STORE / WRITE-BACK phase (memory or register write) — located in the registers and/or main memory. The result is written back to a register or stored to memory, and status flags (e.g. zero, carry) in the status/flag register are updated. The cycle then returns to FETCH for the next instruction.

So the cycle is drawn as: FETCH (CU + PC/MAR/MDR + Memory) → DECODE (CU/decoder) → EXECUTE (ALU + registers) → STORE/WRITE-BACK (registers/Memory) → back to FETCH.

(b) Description of each phase.

Fetch: The CPU reads the next instruction from main memory. The contents of the PC (address of the next instruction) are loaded into the MAR and sent on the address bus; memory returns the instruction word on the data bus into the MDR and then into the IR. The PC is incremented so it points to the following instruction.

Decode: The Control Unit examines the opcode held in the IR and determines what operation is to be performed and which operands/registers are needed. It then asserts the appropriate control signals to the ALU, registers and memory for the execute phase.

Execute: The operation is actually carried out. The ALU performs arithmetic or logical operations on operands taken from registers or memory, or the instruction performs a data transfer or a branch that changes the PC. The result and status flags are produced.

Store/Write-back: The result of execution is written back to a destination register or to main memory, and the condition/status flags are updated. Control then passes back to the fetch phase to begin the next machine cycle.

Unofficial marking guide:

- *1 mark: correct identification and location of the FETCH phase (Control Unit, PC/MAR/MDR, main memory)*
- *1 mark: correct identification and location of the DECODE phase (Control Unit / instruction decoder, IR)*
- *1 mark: correct identification and location of the EXECUTE phase (ALU and registers under CU control)*
- *1 mark: correct identification and location of the STORE/WRITE-BACK phase (registers and/or main memory)*
- *1 mark: cycle drawn as a complete loop returning from store/write-back to fetch, with phases in correct order*
- *1 mark: clear, labelled diagram showing the flow/arrows between phases*
- *1 mark: accurate description of the fetch phase (PC→MAR→memory→MDR→IR, PC incremented)*
- *1 mark: accurate description of the decode phase (opcode decoded, control signals generated)*
- *1 mark: accurate description of the execute phase (ALU/registers perform the operation)*
- *1 mark: accurate description of the store/write-back phase (result written to register/memory, flags updated)*

Model confidence: medium.

Model note: The question refers to a diagram that is not supplied; the answer assumes the standard four-phase fetch–decode–execute–store machine cycle. Some textbooks merge store into execute (three-phase cycle); if the lecturer's notes use a three-phase model, the store/write-back marks should be folded into execute. 'Exact location' is interpreted as the hardware unit (CU, ALU, registers, memory) in which each phase occurs.

QUESTION A6 (10 marks)

Topic: Data representation and number systems

a) Answer: left 24 -> 0 0011000; right -83 -> 1 1010011; 24 - (-83) = 107 -> sign-magnitude 0 1101011

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

b) Answer: left 24 -> 1's 00011000, 2's 00011000; right -83 -> 1's 10101100, 2's 10101101; 24 - (-83) = 107 -> 1's 01101011, 2's 01101011

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*

© Hak Cipta Universiti Teknologi MARA

• *1 mark for converting the result back to decimal.*

QUESTION A7 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (2).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Registers are small, high-speed storage locations inside the CPU used for temporary data, addresses and control information. Five common types are:

1. Program Counter (PC) / Instruction Pointer: Holds the address of the next instruction to be fetched from memory. After each fetch it is incremented (or updated by a branch/jump) so the CPU knows where to continue execution.

2. Instruction Register (IR): Holds the instruction currently being executed. The fetched instruction is copied into the IR, where the control unit decodes its opcode and operands.

3. Memory Address Register (MAR): Holds the address of the memory location to be accessed (read or written). It connects the CPU to the address bus during a memory operation.

4. Memory Data Register (MDR) / Memory Buffer Register (MBR): Holds the data being transferred to or from memory. On a read, the fetched word is placed in the MDR before being copied to another register; on a write, the data to be stored is placed in the MDR.

5. Accumulator (AC): A general-purpose arithmetic/logic register that holds one operand before an ALU operation and receives the result afterwards. It is also used for implicit input/output and shift operations in accumulator-based machines.

(Other acceptable examples include general-purpose registers, status/flag register, stack pointer, index register and temporary registers.)

Unofficial marking guide:

• *1 mark: correct name of a register type*
• *1 mark: correct description of its function/role*
• *1 mark: correct name of a second register type*
• *1 mark: correct description of its function/role*
• *1 mark: correct name of a third register type*
• *1 mark: correct description of its function/role*
• *1 mark: correct name of a fourth register type*
• *1 mark: correct description of its function/role*
• *1 mark: correct name of a fifth register type*
• *1 mark: correct description of its function/role*

Model confidence: high.

Model note: The question asks for FIVE types; the model answer lists five with one mark each for name and function. Other valid register types (general-purpose, status/flags, stack pointer, index) should be accepted. No figure or table is required.

QUESTION A8 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (12).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two functions of an I/O module (4 marks):

Function 1 — Interface/control and timing (device communication): The I/O module acts as an intermediary between the CPU/system bus and the external device. It converts the CPU's commands and data formats into signals the device can understand (and vice versa), and it synchronises the transfer with the device's timing, buffering data so that the fast CPU and slow peripherals can operate together. In effect it hides device-specific details from the processor.

Function 2 — Data buffering and error/status handling: The module temporarily stores (buffers) data during a transfer to compensate for speed differences between CPU, memory and peripheral, and it performs error detection/reporting and status reporting (e.g. ready/busy, error flags) back to the CPU. It may also relieve the CPU of low-level tasks such as data formatting and, in DMA-capable modules, direct memory transfers.

(b) Two categories of external I/O devices with examples (4 marks):

© Hak Cipta Universiti Teknologi MARA

Category 1 — Human-readable (human–machine interface) devices: devices that communicate with the user, e.g. keyboard, mouse, monitor/display, printer.

Category 2 — Machine-readable devices: devices that communicate with other equipment/machines, e.g. disk drives, magnetic tape drives, sensors/actuators, network interface cards.

(The third standard category is communication devices, e.g. modem, network card.)

(c) I/O techniques (2 marks):

Requires active CPU intervention: Programmed I/O (polling) — the CPU repeatedly checks the device status and moves each data item itself. (Interrupt-driven I/O also involves the CPU per transfer, but programmed I/O is the clearest example.)

Does not require active CPU intervention: Direct Memory Access (DMA) — the DMA controller transfers data directly between the device and memory, interrupting the CPU only on completion.

Unofficial marking guide:

• 1 mark: correct identification of a first I/O module function (e.g. interface/control and timing between CPU and device)
• 1 mark: discussion/explanation of that first function
• 1 mark: correct identification of a second I/O module function (e.g. data buffering, error/status handling)
• 1 mark: discussion/explanation of that second function
• 1 mark: correct naming of one device category (e.g. human-readable)
• 1 mark: appropriate example for that category (e.g. keyboard/monitor)
• 1 mark: correct naming of a second device category (e.g. machine-readable)
• 1 mark: appropriate example for that category (e.g. disk drive/sensor)
• 1 mark: names an I/O technique requiring active CPU intervention (programmed/polling I/O)
• 1 mark: names an I/O technique not requiring active CPU intervention (DMA)

Model confidence: high.
Model note: The question text is complete and needs no figure. For part (c), interrupt-driven I/O could also be argued as CPU-intervention-based; the model answer uses programmed I/O as the clearest example and notes DMA as the no-intervention technique. The third device category (communication devices) is mentioned only for completeness and is not required by the question.

QUESTION A9 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (6).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The input/output problem occurs because the CPU and I/O devices differ greatly in speed, data format and control method, so the processor cannot simply connect directly to peripherals. There is a wide speed mismatch (the CPU runs in nanoseconds while peripherals such as disks or keyboards operate in milliseconds), devices use different data formats and word lengths, and each device has its own control logic and operating characteristics. Therefore an I/O module is needed to act as an interface that synchronises, formats and controls the transfer of data between the CPU/memory and the peripherals. [1 mark]

(b) I/O modules can be classified into three categories according to their function and capability:

1. Human-readable / user-interface module: This category interfaces the computer to the human operator. It converts the user's actions into digital signals the computer can process and presents computer output in a form humans can understand. Example: keyboard, mouse, monitor (VDU) or printer. [3 marks]

2. Machine-readable module: This category interfaces the computer to equipment that reads or writes machine-encoded data, typically for secondary storage or data transfer. It handles blocks of data rather than single characters. Example: magnetic disk drive, magnetic tape drive, or optical disk (CD/DVD) drive. [3 marks]

3. Communication module: This category interfaces the computer to a remote device or network, allowing data to be exchanged over a communication link. It performs serial/parallel conversion, formatting and error control. Example: a network interface card (NIC), modem, or serial communication port. [3 marks]

(Total for (b) = 9 marks; total question = 10 marks.)

Unofficial marking guide:

• 1 mark: (a) correct explanation of the I/O problem (speed mismatch / format differences / need for an interface

module).

• 3 marks: (b) first category correctly named and described (1 mark) with an appropriate example (1 mark) and clear explanation of its role (1 mark).

• 3 marks: (b) second category correctly named and described (1 mark) with an appropriate example (1 mark) and clear explanation of its role (1 mark).

• 3 marks: (b) third category correctly named and described (1 mark) with an appropriate example (1 mark) and clear explanation of its role (1 mark).

Model confidence: high.

Model note: The question text does not specify the exact classification scheme; the answer uses the standard Stallings classification of I/O modules as human-readable, machine-readable and communication. Award marks for any valid equivalent three-category classification with correct examples.

QUESTION A10 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (5).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Direct mapping does not need a replacement algorithm. In a direct-mapped cache each main-memory block can be placed in exactly one cache line (line = block address mod number of lines), so when a new block must be loaded there is no choice of victim: the block currently occupying that unique line is simply overwritten. No decision policy is required. (By contrast, fully associative and set-associative caches allow a block to go into more than one line, so a replacement algorithm is needed to choose which line to evict.)

(b) Four types of replacement algorithm:

1. Random replacement: the line to be evicted is chosen at random (often by a pseudo-random counter). It is very simple and cheap to implement in hardware and needs no bookkeeping of usage, but it ignores locality and so generally gives poorer hit ratios than usage-based schemes.

2. FIFO (First-In, First-Out): the line that has been resident in the cache (or set) the longest is evicted first, tracked by a circular queue/age counter per set. It is simple to implement but suffers from the fact that a frequently used block loaded long ago may be evicted even though it is still hot (no regard for actual usage).

3. LRU (Least Recently Used): the line that has not been referenced for the longest time is evicted, on the principle that recently used blocks are likely to be used again (temporal locality). It gives good hit ratios but requires tracking of reference order (e.g. counters or a stack), which is costly in hardware, especially for large associativity.

4. LFU (Least Frequently Used): the line with the smallest number of references (access count) is evicted, on the principle that heavily used blocks should be retained. It requires a counter per line and has the drawback that blocks that were hot in the past but are no longer used keep high counts and are not evicted (cache pollution); counters also need periodic resetting.

(Other acceptable variants, e.g. NRU/clock or optimal/OPT, may be credited if described.)

Unofficial marking guide:

• 2 marks: (a) correctly identifies direct mapping as needing no replacement algorithm

• 2 marks: (b) each of four replacement algorithms correctly named (0.5 mark each)

• 4 marks: (b) each algorithm given a valid description of how it selects the victim (1 mark each)

• 2 marks: (b) overall accuracy/quality of descriptions (e.g. noting hardware cost or hit-ratio trade-off) across the four algorithms

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. Mark allocation for (b) is split between naming and describing each algorithm; accept equivalent valid algorithms (e.g. NRU/clock, OPT) if properly described.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (17).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 2 reads y = A(A + 2B/C-D 2). The only consistent reading with the available instructions (no exponentiation) is y = A × (A + 2B/C − D²), i.e. the trailing 2 is a superscript square on D. This is assumed throughout; if the figure shows a different expression, the operand order changes but the method is identical.

Sub-expression plan (common to all three programs): compute T1 = 2B/C, T2 = D×D, T3 = A + T1 − T2, then y = A × T3. A, B, C, D are read-only; x, y, z are temporaries; final result in y.

Zero-address program (stack machine):

```
PUSH A        ; stack: A
    PUSH B        ; stack: A, B
    PUSH B        ; stack: A, B, B
    ADD           ; stack: A, 2B       (B+B)
    PUSH C        ; stack: A, 2B, C
    DIV           ; stack: A, 2B/C
    ADD           ; stack: A + 2B/C
    PUSH D        ; stack: (A+2B/C), D
    PUSH D        ; stack: (A+2B/C), D, D
    MUL           ; stack: (A+2B/C), D*D
    SUB           ; stack: A + 2B/C - D*D
    PUSH A        ; stack: (A+2B/C-D*D), A
    MUL           ; stack: A*(A+2B/C-D*D)
    POP y         ; store final answer in y
```

One-address program (accumulator machine):

```
LOAD B        ; ACC = B
    ADD B         ; ACC = 2B
    DIV C         ; ACC = 2B/C
    ADD A         ; ACC = A + 2B/C
    STOR x        ; x = A + 2B/C   (save partial result)
    LOAD D        ; ACC = D
    MUL D         ; ACC = D*D
    STOR z        ; z = D*D
    LOAD x        ; ACC = A + 2B/C
    SUB z         ; ACC = A + 2B/C - D*D
    MUL A         ; ACC = A*(A + 2B/C - D*D)
    STOR y        ; y = final answer
```

Three-address program (register/operand form ADD(p,q,r) meaning p = q op r):

```
ADD (x, B, B)     ; x = 2B
    DIV (x, x, C)     ; x = 2B/C
    ADD (x, A, x)     ; x = A + 2B/C
    MUL (z, D, D)     ; z = D*D
    SUB (x, x, z)     ; x = A + 2B/C - D*D
    MUL (y, A, x)     ; y = A*(A + 2B/C - D*D)  final answer in y
```

Comment on correctness: In every version the read-only registers A, B, C, D are only ever read as source operands, the temporaries x and z hold intermediate values, and the final result is written to y, satisfying the stated constraints.

Unofficial marking guide:

• 4 marks: zero-address program — correct use of PUSH/POP with stack semantics, correct operand order for DIV and SUB, and final POP into y

• 4 marks: one-address program — correct LOAD/STOR/ADD/SUB/MUL/DIV sequence, correct use of temporaries x and z, final STOR y

© Hak Cipta Universiti Teknologi MARA

• 4 marks: three-address program — correct ADD/DIV/MUL/SUB with (p,q,r) destination-first convention, correct final MUL into y

*• 4 marks: correct evaluation of the expression (2B/C, D*D, A + 2B/C − D*D, then multiply by A) in all three programs*

• 2 marks: comments and correct assembly notation throughout, with A, B, C, D unmodified and final answer in y

• 2 marks: overall clarity, consistent register/temporary usage and no redundant or missing operations

Model confidence: medium.

Model note: The equation in Figure 2 is OCR-ambiguous ('D 2'); assumed to be D squared since no exponent instruction exists. The three-address table shows 'MUL (p.q.r)' — read as MUL(p,q,r). The (p,q,r) convention is assumed to be destination-first (p = q op r); if the lecturer uses source-first ordering, swap the operand positions accordingly. Figure 2 itself was not available beyond the extracted text.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA